“Cross-Platform Sentiment Analysis”

Submitted by:

Ajay Juyal

Under the guidance of

Mr Neeraj Panwar

(Assistant Professor)

Graphic Era Hill University, Dehradun

Bachelor Of Computer Applications

School Of Computing

Department Of Computer Applications

Date:10/06/2025

CERTIFICATE

This is to certify that the thesis titled “Cross-Platform Sentiment Analysis” submitted by Ajay Juyal, to Graphic Era Hill University for the award of the degree of Bachelor of Computer Application, is a bona fide record of the research work done by him under our supervision. The contents of this project in full or in parts have not been submitted to any other Institute or University for the award of any degree or diploma.

Neeraj Panwar

(Assistant Professor)

GEHU,Dehradun

Place: Dehradun

Date: 10/06/2025

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to Mr. Neeraj Panwar, my professor, for his invaluable support and guidance throughout this project. His expertise, encouragement, and constructive feedback have been instrumental in successfully completing this work.

This project has been a tremendous learning experience, and I am deeply thankful for the opportunity to work under his mentorship.

Thank you.

Ajay Juyal

07

2221095

ABSTRACT

In the digital age, public sentiment plays a critical role in shaping trends, influencing public opinion, and guiding business decisions. Social media platforms such as Tumblr, YouTube, and Reddit generate vast amounts of user-generated content that reflect real-time public sentiment. This project, titled "Cross-Platform Sentiment Analysis," aims to harness the potential of live data from these platforms to provide meaningful insights through sentiment analysis.

The core objective of this project is to develop a real-time system that collects data using the official APIs of Tumblr, YouTube, and Reddit, analyzes the sentiment expressed in this content using Natural Language Processing (NLP) techniques, and presents the results through an interactive and intuitive Streamlit dashboard. The project leverages the VADER (Valence Aware Dictionary for Sentiment Reasoning) model—a lexicon and rule-based sentiment analysis tool optimized for social media text—to classify sentiments into positive, negative, or neutral categories.

A key feature of this project is its ability to work with live data, automatically fetching recent posts, comments, and discussions relevant to user-defined keywords or topics. The system then processes and visualizes the sentiment distribution across all three platforms, enabling users to observe real-time public opinion dynamics.

For scalability, reproducibility, and continuous integration/deployment (CI/CD), the entire project is Dockerized and deployed on AWS EC2, with automated build and deployment pipelines configured using Jenkins. This ensures that the system remains robust, secure, and maintainable over time.

The project not only demonstrates technical proficiency in data engineering, NLP, and full-stack deployment but also serves as a valuable tool for researchers, marketers, and analysts who need to track cross-platform sentiment trends in real-time.

TABLE OF CONTENTS

1. Introduction……6

1.1. Introduction to stay-Hub
1.2. Features
1.3. Structure

2. Implementation……13

2.1. System Requirements
2.2. Methodologies
2.3. Tools and Technologies
2.3.1. Programming Language
2.3.2. Data Analysis and Sentiment Modeling
2.3.3. APIs and Data Sources
2.3.4. Dashboard and UI
2.3.5. Containerization and Deployment
2.3.6. CI/CD Automation
2.3.7. Authentication and Enviroment Management
2.3.8. Package and Dependency Management
2.3.9. Testing Tool
2.3.10. Code Management

3. Challenges Faced……..20

3.1. Real-Time Challenges
3.2. Technologies Related Challenges

4. Future Enhancements……..24

4.1. Updates and feedback

5. Conclusion and Future Scope……..26

5.1. Conclusion
5.2. How it will make impact in market

6. Appendix……...29

6.1. Code Snippet
6.2. Snapshot

7. References……37

7.1. Documentation and official resources

INTRODUCTION

1.1 Introduction to Cross-Platform Sentiment Analysis

With the exponential growth of digital content, social media platforms have emerged as powerful tools for communication, opinion sharing, and information dissemination. Platforms such as Tumblr, YouTube, and Reddit host millions of user interactions daily, capturing public sentiments on a wide range of topics—from politics and entertainment to technology and global events. Analyzing these sentiments provides valuable insights into public opinion trends and collective human behavior.

Sentiment analysis, also known as opinion mining, is a subfield of Natural Language Processing (NLP) that focuses on identifying and categorizing sentiments expressed in textual data. Traditional sentiment analysis models rely on static datasets, limiting their ability to reflect current trends or changes in public opinion. In contrast, this project—Cross-Platform Sentiment Analysis—focuses on gathering real-time data from multiple social media platforms and analyzing it dynamically to provide live sentiment tracking.

The primary objective of the project is to build a system that:

- Collects live textual data from Tumblr, YouTube, and Reddit using their official APIs,
- Processes this data using the VADER sentiment analysis model, which is particularly well-suited for short social media texts and slang,
- Displays the sentiment results in a user-friendly Streamlit dashboard,
- And ensures scalability and automation by deploying the application on AWS EC2 with Docker containers and Jenkins for continuous integration and deployment (CI/CD).

The VADER (Valence Aware Dictionary for Sentiment Reasoning) model is a lexicon and rule-based sentiment analysis tool designed specifically for sentiments expressed in social media. It is capable of detecting sentiment polarity (positive, negative, neutral) as well as the intensity of sentiment in the given text. By applying VADER to live social media content, the project can reveal how users across different platforms feel about any particular topic in real-time.

To facilitate visualization and interaction, a Streamlit dashboard was developed. This dashboard allows users to input keywords or topics of interest, view real-time sentiment distribution across the three platforms, and monitor changes in sentiment over time. It bridges the gap between raw data and actionable insights, making the analysis accessible even to non-technical users.

Furthermore, to ensure that the application is maintainable, scalable, and deployable across different environments, Docker is used to containerize the application. The entire pipeline is deployed on an AWS EC2 instance, providing a reliable and secure cloud infrastructure. Jenkins is integrated to automate the build, test, and deployment processes, forming a complete CI/CD pipeline for efficient development and delivery.

In conclusion, this project showcases the integration of live data gathering, NLP-based sentiment analysis, interactive dashboards, and full-stack DevOps practices. It serves as a real-world application of machine learning, cloud computing, and software engineering principles—providing a comprehensive view of cross-platform public sentiment in real time.

1.2 Features

he Cross-Platform Sentiment Analysis project is built to be practical, user-friendly, scalable, and insightful. It integrates data science, web technologies, and DevOps to deliver a complete end-to-end solution. Below is a breakdown of its most important features:

Live Data Collection from Multiple Platforms

- **APIs Used:** Official APIs from Tumblr, YouTube, and Reddit.
- **Functionality:** Users can input a keyword or topic, and the system fetches real-time data (posts, comments, video titles, etc.) from all three platforms.
- **Purpose:** Enables the system to reflect the current public sentiment rather than relying on outdated or static datasets.

Cross-Platform Sentiment Comparison

- **Unique Angle:** Unlike most sentiment analysis tools that focus on one platform, this project analyzes sentiment across Tumblr, YouTube, and Reddit simultaneously.
- **Advantage:** Users get a more diverse and comprehensive view of public opinion, helping to identify differences in how each platform perceives the same topic.

Sentiment Analysis Using VADER

- **Model Used:** VADER (Valence Aware Dictionary and Sentiment Reasoner), a rule-based model optimized for social media content.
- **Sentiment Classes:** The model classifies text into:
    - Positive
    - Negative

    - Neutral
- **Output:** Also provides a compound score representing the overall sentiment intensity.
- **Why VADER:** Ideal for short-form, informal, and emoji-rich social media content. It handles slang, emojis, negations, and capitalization better than many traditional models.

Interactive Streamlit Dashboard

- **UI/UX:** Built using Streamlit, a fast and modern Python web framework.
- **Features:**
    - User inputs for keyword/topic
    - Live data display (platform-wise)
    - Real-time sentiment breakdown (positive, negative, neutral)
    - Bar charts, pie charts, or time-series graphs for easy interpretation
    - Clean and responsive interface
- **Usefulness:** Makes complex analysis accessible to non-technical users such as marketers, content creators, and researchers.

Real-Time Updates and Auto Refresh

- **Functionality:** The dashboard can be set to refresh automatically to update sentiment scores with new data.
- **Advantage:** Keeps users informed with the most current sentiment trends, crucial during live events (e.g., elections, product launches, controversies).

Scalable & Portable via Docker

- **Dockerization:** The entire application (data collection, processing, and dashboard) is containerized using Docker.
- **Benefits:**
    - Easy to deploy on any system (cross-platform)
    - Isolation of dependencies
    - Simplifies environment setup and ensures consistency

Cloud Deployment on AWS EC2

- **Hosting:** The application runs on an Amazon EC2 instance, providing reliable uptime and access over the web.
- **Advantages:**

    - Scalability and flexibility
    - Secure and cost-efficient infrastructure
    - Accessible anywhere via a public IP or domain

Continuous Integration/Deployment with Jenkins

- **CI/CD Pipeline:**
    - Jenkins automates code build, test, and deployment processes.
    - Every code push or update triggers the pipeline to rebuild and redeploy the app on EC2.
- **Benefits:**
    - Reduces human error
    - Speeds up development iterations
    - Keeps the production environment up-to-date automatically

Secure API Key Handling

- **Security Practice:** All API keys and secrets are stored using environment variables or .env files.
- **Purpose:** Prevents accidental exposure of sensitive credentials in source code or version control (e.g., GitHub).

Modular and Extensible Design

- **Architecture:** The system is built with modularity in mind—each component (data collection, analysis, dashboard, deployment) can be updated or replaced independently.
- **Future Proofing:**
    - Easy to integrate new platforms like Twitter/X, Instagram, or LinkedIn
    - Can be upgraded to use deep learning models like BERT or RoBERTa

1.3 Structure

The Cross-Platform Sentiment Analysis project is structured in a modular, maintainable way, with each file and folder responsible for a specific part of the system's functionality. The architecture is organized into four main layers: API Integration, Sentiment Analysis, Dashboard UI, and Deployment & Configuration.

```
project-root/
├── __pycache__/
├── venv/
├── .env
├── .gitignore
├── main.py
├── flask_callback_server.py
├── reddit_api.py
├── tumblr_api.py
├── tumblr_oauth.py
├── youtube_api.py
├── sentiment.py
├── test_reddit.py
├── requirements.txt
├── Dockerfile
└── Jenkinsfile
```

1. __pycache__/

- This directory is automatically created by Python to store the compiled bytecode files of your Python scripts. It improves loading speed but does not contain any source code you edit.

2. venv/

- The virtual environment folder contains a self-contained Python environment with all the libraries and dependencies your project requires. Using a virtual environment avoids conflicts between packages used in different projects.

3. .env

- This file holds sensitive information such as API keys, tokens, and environment-specific variables. It is kept separate from the code to enhance security and is loaded at runtime to configure the app dynamically.

4. .gitignore

- This file lists files and directories that should be ignored by Git version control. Typically, it includes venv/, __pycache__/, .env, and other local or system files that shouldn't be committed to the repository.

5. main.py

- The main entry point of the application. It launches the Streamlit dashboard, integrates all other modules, handles user inputs, and displays the results of sentiment analysis dynamically.

6. flask_callback_server.py

- A minimal Flask web server that handles OAuth callback requests, necessary for authenticating certain APIs (such as Tumblr). It listens for OAuth redirects and manages token exchanges.

7. reddit_api.py

- Contains the logic to connect to Reddit's API, query posts or comments based on user input, and return raw textual data for sentiment analysis.

8. tumblr_api.py

- Handles fetching posts from Tumblr using authenticated API calls. This module relies on tumblr_oauth.py for managing OAuth authentication.

9. tumblr_oauth.py

- Manages the OAuth 1.0a flow required by Tumblr's API, including generating signed requests, managing tokens, and refreshing credentials as needed.

10. youtube_api.py

- Interacts with the YouTube Data API to search for videos and retrieve comments, which are then used as input for sentiment evaluation.

11. sentiment.py

- Implements the core sentiment analysis functionality using the VADER sentiment analysis model. This module processes text data and classifies it into positive, neutral, or negative sentiments with associated scores.

12. test_reddit.py

- A testing script designed to verify the functionality and reliability of the Reddit API integration. It helps ensure that the API connection and data fetching operate as expected.

13. requirements.txt

- Lists all external Python libraries and dependencies that must be installed for the project to run correctly. This file facilitates environment setup and deployment consistency.

14. Dockerfile

- Contains instructions for building a Docker container image that packages the entire application and its dependencies into a portable environment. This ensures that the application runs consistently across different platforms and simplifies deployment.

15. Jenkinsfile

- Defines the Continuous Integration and Continuous Deployment (CI/CD) pipeline used by Jenkins. It automates tasks such as fetching code from version control, running tests, building Docker images, and deploying the application to the AWS EC2 instance.

IMPLEMENTATION

2.1 System Requirements

To successfully build, run, and maintain the Cross-Platform Sentiment Analysis project, ensure your system meets the following requirements:

Hardware Requirements:

1. **Processor:** Dual-core or higher (Intel i3/i5/i7 or AMD equivalent).
2. **RAM:** Minimum 4GB (8GB recommended for optimal performance, especially when running Docker containers).
3. **Storage:** At least 1GB of free disk space for project files, dependencies, and local caching of API data.

Software Requirements:

1. **Operating System:**
    - Local Development: Windows 10/11, macOS, or Linux.
    - Deployment: Ubuntu 20.04 LTS (or higher) on AWS EC2 instance.
2. **Development Environment:**
    - Python 3.8 or higher installed locally and on the EC2 instance.
    - Python package manager (pip) for managing dependencies.
3. **APIs and Libraries:**
    - Reddit API access with developer credentials.
    - YouTube Data API v3 access via Google Cloud API key.
    - Tumblr API access with OAuth credentials.
    - VADER Sentiment Analysis model (installed via Python libraries).
4. **Containerization and Deployment:**
    - Docker installed on local machine and Ubuntu EC2 instance.
    - AWS EC2 instance running Ubuntu for hosting the application.
    - Jenkins installed (locally or on a server) to automate build and deployment pipelines.

5. **Additional Tools:**
    - Code Editor: Vim/Nano editor or Visual Studio Code or any Python-supported IDE.
    - Version Control: Git for source code management.
    - Postman (optional) for API testing.

Dependencies:

- streamlit
- requests
- vaderSentiment
- google-api-python-client
- praw or asyncpraw
- flask
- Other dependencies listed in requirements.txt

Networking Requirements:

1. **Internet Connection:** Required for live API access (Reddit, YouTube, Tumblr) and for connecting to AWS services.
2. **Ports:**
    - Streamlit default port (8501) must be open on EC2 security group.
    - Flask callback server port (e.g., 5000) must also be open for OAuth redirects.

Optional:

- Browsers: Latest Google Chrome, Firefox, or Edge for dashboard access.
- AWS CLI for managing EC2 instance and deployments from command line.

2.2 Methodologies

The methodology outlines the systematic approach taken to develop the Cross-Platform Sentiment Analysis project, including data acquisition, processing, analysis, visualization, and deployment phases. This ensures a robust, scalable, and maintainable application capable of performing real-time sentiment analysis across multiple social media platforms.

Data Acquisition

- **API Integration:**
  The project connects with three major social media platforms — **Reddit**, **YouTube**, and **Tumblr** — via their respective APIs.
  - **Reddit API:** Using the praw (Python Reddit API Wrapper) library to authenticate and fetch posts and comments based on user-provided keywords or subreddits.
  - **YouTube Data API:** Utilizing Google's API client to retrieve video metadata and comments related to specific search queries or channels.
  - **Tumblr API:** Accessing Tumblr posts through authenticated requests using OAuth 1.0a protocol, managed via a custom Flask callback server to handle OAuth token exchanges securely.
- **Real-Time Data Fetching:**
  User inputs keywords or topics via the Streamlit dashboard, triggering live API calls to fetch the most recent data from all three platforms simultaneously. This allows the system to provide up-to-date sentiment insights.

Data Preprocessing

- **Text Extraction:**
  The raw data fetched from APIs consists of post titles, bodies, comments, and metadata. Only relevant text content is extracted for sentiment evaluation.
- **Cleaning:**
  The text data undergoes preprocessing steps such as:
  - Removing URLs, special characters, and emojis.
  - Normalizing case (converting to lowercase).
  - Removing stopwords or irrelevant filler words if needed.

Sentiment Analysis

- **Model Choice:**
  The VADER (Valence Aware Dictionary and sEntiment Reasoner) model is selected for sentiment analysis due to its suitability for social media text, ease of use, and effectiveness in capturing the sentiment of short, informal text.
- **Sentiment Scoring:**
  Each text snippet is analyzed by VADER, which outputs sentiment scores classified as:
  - **Positive**
  - **Neutral**

    - **Negative**
      along with a compound score representing overall sentiment intensity.

- **Aggregation:**
  Sentiment scores from all posts and comments retrieved per platform are aggregated to provide an overall sentiment metric for each platform and combined cross-platform results

Visualization and Dashboard

- **Streamlit Dashboard:**
  The processed sentiment data is visualized in a user-friendly interface built with Streamlit. Features include:
    - Input field for entering keywords/topics.
    - Real-time display of sentiment analysis results for each platform and combined data.
    - Graphs and charts to represent sentiment distribution and trends (e.g., pie charts, bar graphs).
    - Dynamic updates triggered by user interaction without needing to refresh the page manually.

Authentication Handling

- **OAuth for Tumblr:**
  Since Tumblr API requires OAuth 1.0a authentication, a small Flask-based callback server handles OAuth token requests and responses, ensuring secure and seamless authentication without exposing credentials.

Testing and Validation

- **Unit Testing:**
  Individual API modules (e.g., reddit_api.py) include test scripts (test_reddit.py) to verify successful connection, data fetching, and error handling.
- **Integration Testing:**
  Tests ensure that data flows correctly from API fetching to sentiment processing and finally to dashboard visualization.
- **Performance Testing:**
  The system is tested for responsiveness and reliability when fetching large amounts of data or when API rate limits are approached.

Deployment

- **Containerization with Docker:**
  The entire application, including Python dependencies and environment

configurations, is packaged into a Docker container to ensure consistent environments across development and production.

- **AWS EC2 Deployment:**
  The Docker container is deployed on an Ubuntu 20.04 LTS AWS EC2 instance configured with necessary security groups and open ports. This setup enables scalable and remote access to the dashboard.
- **Continuous Integration/Continuous Deployment (CI/CD):**
  Jenkins is configured to automate:
  - Pulling the latest code from the Git repository.
  - Building and testing the Docker image.
  - Deploying the updated container on the EC2 instance.
    This pipeline ensures smooth and efficient updates to the live system with minimal downtime.

2.3 Tools and Technologies

The Cross-Platform Sentiment Analysis project employs a comprehensive range of tools and technologies across software development, data acquisition, sentiment analysis, data visualization, containerization, deployment, and continuous integration. Each component has been carefully selected to ensure reliability, efficiency, scalability, and ease of use throughout the project's lifecycle.

Programming Language

- **Python 3.8 or higher**
  Python is the core programming language used in this project. Its simplicity, readability, and strong ecosystem of libraries make it ideal for building API integrations, handling data, performing sentiment analysis, and constructing the Streamlit dashboard.

Data Analysis and Sentiment Modeling

- **VADER (Valence Aware Dictionary and Sentiment Reasoner)**
  VADER is a rule-based sentiment analysis tool specifically tuned for sentiments expressed in social media content. It is lexicon-based and does not require any machine learning model training. VADER provides polarity scores that classify the input text as positive, neutral, or negative along with a compound score that captures overall sentiment intensity.
- **Pandas and NumPy (used optionally)**
  These are Python libraries that may be used to handle tabular data and perform

numerical operations efficiently. Pandas is useful for data organization, filtering, and aggregation, while NumPy enhances performance for numerical computations.

APIs and Data Sources

- **Reddit API (via PRAW)**
  The project uses Reddit's official API through the PRAW (Python Reddit API Wrapper) library to search, retrieve, and parse Reddit posts and comments. The API allows keyword-based searches and data extraction for further sentiment analysis.
- **YouTube Data API v3**
  This API is part of Google Cloud's platform and enables access to YouTube video metadata and comment threads. The project retrieves video titles, descriptions, and top-level comments related to a user-provided keyword or topic.
- **Tumblr API**
  Tumblr provides a RESTful API for accessing blog posts and metadata. Since Tumblr uses OAuth 1.0a for authentication, the application includes a custom OAuth flow using Flask to securely obtain access tokens required for data retrieval.

Dashboard and User Interface

- **Streamlit**
  Streamlit is a Python-based open-source framework that simplifies the process of building interactive web applications and dashboards. In this project, it is used to allow users to enter keywords, trigger API requests, and view sentiment analysis results in real time with visualizations like bar charts and pie charts. It also supports auto-refresh features for dynamic updates.

Containerization and Deployment

- **Docker**
  Docker is used to package the entire application, including source code, dependencies, and environment variables, into isolated containers. This ensures that the application runs consistently across different machines and eliminates issues caused by dependency mismatches. Docker simplifies testing, deployment, and maintenance.
- **AWS EC2 (Ubuntu Instance)**
  The project is deployed on an Ubuntu 20.04 LTS instance hosted on Amazon Web Services (AWS) Elastic Compute Cloud (EC2). The instance serves as a cloud server for running the Dockerized application and exposes the Streamlit dashboard over the internet. The Ubuntu environment is lightweight, secure, and compatible with Docker and Python-based applications.

CI/CD Automation

- **Jenkins**
  Jenkins is an automation server used to implement Continuous Integration and Continuous Deployment (CI/CD) pipelines. In this project, Jenkins automatically pulls updated code from the Git repository, builds a new Docker image, runs tests, and

redeploys the application to the EC2 instance. This automation reduces manual errors, improves deployment speed, and maintains consistency across versions.

Authentication and Environment Management

- **Flask**
  Flask is a lightweight web framework used in this project to implement a callback server for Tumblr's OAuth authentication. This server handles redirection and token exchanges required for secure API access. It ensures sensitive OAuth credentials are never exposed in the front end.
- **dotenv and .env Files**
  Environment variables such as API keys, client secrets, and access tokens are stored in a .env file. The python-dotenv package loads these variables into the application securely. This method avoids hardcoding sensitive information into the codebase and supports better configuration management.

Package and Dependency Management

- **pip and requirements.txt**
  The Python package manager pip is used to install and manage dependencies. The requirements.txt file lists all the packages and their versions required to run the project. This ensures that the environment is reproducible and consistent across machines.
- **Virtualenv**
  Virtual environments isolate project-specific dependencies from the global Python environment. This helps prevent version conflicts between different Python projects and keeps the development environment clean.

Testing Tools

- **Postman**
  Postman is used during development to test API endpoints, send custom HTTP requests, and debug issues. It helps in validating that the API responses are correct before integrating them into the code.
- **Custom Python Test Scripts**
  The project includes test scripts like test_reddit.py that help verify the functionality of individual modules. These tests confirm that data can be fetched correctly from APIs and that the application behaves as expected.

Code Management

- **Git and GitHub**
  Git is used for version control, enabling collaborative development and maintaining a history of changes. GitHub serves as the remote repository for hosting source code and integrates with Jenkins for automated deployment. Version control ensures traceability, facilitates rollbacks, and improves collaboration across team members

CHALLENGES FACED

3.1 Real Time Challenges

Real-time data processing refers to collecting, analyzing, and presenting data with minimal delay. This project aimed to fetch live data from Reddit, YouTube, and Tumblr, perform sentiment analysis on it, and display the results immediately on a Streamlit dashboard. Implementing this functionality presented several real-time challenges:

API Rate Limiting and Throttling

Every platform involved—Reddit, YouTube, and Tumblr—imposes strict API rate limits to prevent misuse and server overload. These limits define how many requests can be made in a set time window, such as per minute or per day. During real-time sentiment analysis, frequent data requests were necessary to ensure updated insights, but hitting these rate limits often caused the APIs to block or delay responses. This disrupted the user experience, as the dashboard either showed outdated data or none at all. To overcome this, request throttling was implemented to pace the number of API calls within acceptable limits. Retry logic helped manage temporary blocks, and instead of fetching large data sets occasionally, the system fetched smaller data sets more frequently. This balance maintained a steady flow of live data without exceeding limits.

Delayed API Responses

In some instances, the APIs—especially from Tumblr and YouTube—returned data with noticeable latency. These delays were unpredictable and depended on network conditions and platform load. Such delays had a direct impact on the dashboard, causing it to appear frozen or unresponsive while waiting for API responses. This diminished the effectiveness of the real-time experience, as users had to wait longer than expected for updated sentiments. To resolve this, asynchronous API calls were used, allowing the dashboard to remain interactive while waiting for data. In addition, Streamlit's st.cache_data feature was used to store recent results temporarily. This caching, combined with timers, ensured that the most recent available data could be displayed even when the current fetch was delayed, creating a smoother user experience.

Varying Data Availability

The availability of content for analysis varied greatly depending on the keyword entered. For some topics, Reddit might yield thousands of results, while YouTube or Tumblr might return

very little or nothing at all. This inconsistency affected visualizations, causing blank sections or skewed sentiment distributions that could mislead users. The challenge here was not technical failure, but platform-specific data scarcity in real time. The solution involved developing a more intelligent system that could detect such disparities and adapt accordingly. If a platform had no new content, fallback messages were shown, and default placeholder data was used to maintain the structure of the dashboard. The refresh logic was also adjusted so that the dashboard could intelligently wait longer for data from slower or less responsive platforms.

Data Parsing and Standardization

The project involved processing data from three completely different platforms, each with its own data schema. Reddit provides a mix of posts and nested comments, YouTube offers video metadata along with comment threads, and Tumblr shares textual blog content and associated metadata. Parsing and normalizing this unstructured data into a consistent format suitable for sentiment analysis proved to be a significant challenge. Real-time analysis required that this conversion be both fast and accurate. Separate parsing and formatting scripts were written for each platform to extract relevant content, standardize the fields (e.g., text, timestamp, platform name), and prepare the data for uniform processing by the VADER sentiment analysis model. This modular approach ensured real-time compatibility while maintaining data integrity.

Auto-Refreshing the Dashboard

Streamlit, while excellent for building data dashboards, does not support background tasks or continuous auto-refresh natively. This posed a challenge for displaying live sentiment results because users had to manually refresh the page to get updated insights. A real-time dashboard is expected to reflect data changes instantly or at regular intervals without user intervention. To address this, Streamlit's session state and experimental rerun features were utilized. Countdown timers were added to automatically refresh the dashboard after a set interval, simulating a real-time experience. While not a perfect substitute for true background execution, this workaround allowed the dashboard to feel responsive and dynamic, aligning with the goals of real-time sentiment tracking.

3.2. Technologies Related Challenges

Building this cross-platform project required integrating multiple APIs, managing various technologies, and ensuring compatibility in deployment. Several technology-specific challenges were encountered:

OAuth Authentication Complexity

Tumblr's API requires OAuth 1.0a for authentication, which is significantly more complex compared to the token-based approaches used by Reddit and YouTube. Unlike OAuth 2.0 or simple API key methods, OAuth 1.0a involves creating a multi-step authentication process that includes generating signatures, handling token requests, and managing user authorization callbacks. This complexity became an obstacle, especially in a frontend-oriented project with minimal backend infrastructure. To address this, a minimal Flask-based callback server was developed specifically for handling the OAuth token flow. This server facilitated secure token exchanges and maintained user sessions during Tumblr data retrieval.

Environment and Dependency Conflicts

Integrating various technologies such as Streamlit, Flask, OAuth libraries, Pandas, and requests created a challenging environment for dependency management. Some packages had conflicting requirements or behaved differently under different Python environments. These conflicts often caused unexpected runtime errors or incompatibilities that hindered development and deployment, especially in a Dockerized setup. To manage this, exact versions of all required packages were defined in the requirements.txt file, and each dependency was tested within a virtual environment. This practice ensured reproducibility and eliminated version mismatch issues when running the project in containers or on cloud infrastructure.

Secure API Key Handling

The project required multiple API credentials for Reddit, YouTube, and Tumblr. Storing these keys insecurely in source files posed a risk of accidental exposure, especially when using Git or deploying to cloud services like AWS EC2. Exposing such credentials could lead to unauthorized usage and breach of API quotas. To ensure security, API keys were stored in .env files during local development, which were explicitly ignored in Git using a .gitignore rule. In production, the environment variables were set directly on the EC2 instance. This dual approach ensured keys remained confidential and were appropriately accessible depending on the environment.

Dockerizing a Multi-Module Project

The project included several interconnected Python modules such as API handlers, the sentiment analysis logic, and the Streamlit frontend. Dockerizing this architecture proved difficult because of the need to maintain proper file paths and ensure that all dependencies and modules were available in the container. Early builds often failed due to unresolved imports or missing files. To solve this, a carefully structured Dockerfile was created, setting the correct working directory and copying all necessary files. The image was built to be modular and self-contained, ensuring all scripts could interact seamlessly once the container was running.

Jenkins Deployment Configuration

Automating the build and deployment process with Jenkins introduced complexity in managing the continuous integration and delivery pipeline. Deploying to an AWS EC2 instance required configuring secure SSH access, handling environment variables, and writing scripts to build and run Docker containers remotely. If any part of this setup—such as permission settings or port forwarding—was misconfigured, the entire pipeline could fail. Jenkins was configured to use SSH key pairs, and scripted build jobs were created to automate the entire process, from pulling the latest code to rebuilding the Docker image and restarting the container on the EC2 instance.

Limited EC2 Instance Resources

The project was deployed on a free-tier Ubuntu-based EC2 instance, which came with limited CPU and RAM resources. Running multiple processes simultaneously—such as fetching data from three APIs, performing sentiment analysis, and rendering visuals on Streamlit—stressed the server's resources. This often led to crashes, slow responses, or memory overflows. To mitigate this, the data load per API call was reduced, and memory-optimized logic was implemented. The system was configured to fetch smaller datasets at more frequent intervals and avoid unnecessary resource consumption, making it sustainable within the free-tier limits.

Visualization and Data Binding in Streamlit

Displaying data from Reddit, YouTube, and Tumblr together in one dashboard introduced visual and structural challenges. Differences in data volume or structure from each platform could lead to unbalanced or cluttered displays. For example, if one API failed or returned too much data, it could distort the layout or overwhelm the interface. To handle this, visualization components were modularized using Streamlit’s containers and columns. Each platform’s data was displayed in its own structured section, with independent charts and summaries. This modular design preserved layout consistency, improved readability, and ensured that the failure or absence of one data source didn’t affect the entire dashboard.

Future Enhancements

4.1 Updates and Feedback

Integration with More Platforms

Currently, the project focuses on Reddit, YouTube, and Tumblr for live sentiment data. Expanding to additional social media platforms such as Twitter (X), Facebook, Instagram, and LinkedIn would greatly increase the breadth and diversity of data. Each platform caters to different user demographics and content types, providing richer, more comprehensive sentiment insights. However, integrating these platforms requires understanding and managing different API protocols, data formats, and rate limits. This expansion would enable users to capture trends and opinions from a wider audience, making the analysis more representative of public sentiment.

Enhanced Sentiment Analysis with NLP Models

The existing VADER model is rule-based and specifically optimized for social media text, offering quick and reasonably accurate sentiment classification. Nonetheless, VADER struggles with complex linguistic features such as sarcasm, irony, or nuanced context. Integrating transformer-based Natural Language Processing (NLP) models like BERT, RoBERTa, or DistilBERT would improve the system's ability to understand context and semantics deeply. These models leverage large-scale pretraining and fine-tuning to better capture subtle sentiment cues, thereby increasing accuracy, especially on informal or complex content. However, they also require more computational resources, which would affect deployment and performance considerations.

User Authentication and Personalization

Adding user authentication will enable personalized experiences, allowing users to create accounts and save preferences such as favorite keywords or topics for monitoring. Users could access historical sentiment trends tailored to their interests and receive notifications via email or in-app alerts when significant changes or spikes in sentiment occur. Personalization enhances user engagement and makes the platform more useful for businesses, marketers, or researchers by providing relevant, timely insights specific to their needs.

Advanced Visualization and Analytics

While the current dashboard provides basic visualizations, adding more sophisticated analytics tools will enhance user understanding of the data. Time series graphs can show sentiment trends over days or weeks, while word clouds highlight frequently mentioned terms in posts and comments. Topic modeling can cluster discussions into themes, helping users identify underlying conversations driving sentiment. Comparative graphs that break down sentiment by platform or region provide more granular insights. Libraries like Plotly, Altair, or D3.js enable highly interactive and visually appealing analytics, making the data easier to interpret and act upon.

Historical Data Archiving and Analytics

At present, the system focuses on live data without storing historical sentiment information. By implementing persistent storage for collected data—using databases or cloud storage—the platform can offer users the ability to view and analyze sentiment trends over time. This historical data supports deeper analytics, such as identifying cyclical patterns, measuring campaign impacts, or training more sophisticated machine learning models. It adds significant value for strategic decision-making and research purposes.

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The Cross-Platform Sentiment Analysis project represents a significant step toward harnessing the power of social media data to gain real-time insights into public opinion and sentiment across multiple platforms. By integrating APIs from Reddit, YouTube, and Tumblr, the system successfully collects live data streams, processes them using the VADER sentiment analysis model, and presents the analyzed results on an intuitive and interactive Streamlit dashboard. This multi-platform approach not only broadens the scope of sentiment monitoring but also enhances the accuracy and reliability of the insights by capturing diverse perspectives from various online communities.

The development of this project involved overcoming numerous technical challenges, including handling API rate limits and throttling, managing OAuth authentication workflows particularly for Tumblr, normalizing disparate data formats, and implementing efficient asynchronous data fetching to maintain real-time performance. These challenges were met with effective engineering solutions such as request throttling, retry mechanisms, creation of a callback server for OAuth, and data normalization scripts that convert heterogeneous data into a consistent format suitable for sentiment analysis.

Deployment considerations also played a crucial role in the project's success. Containerizing the application using Docker ensured environment consistency and simplified the deployment process on an Ubuntu-based AWS EC2 instance. The integration of Jenkins for continuous integration and deployment (CI/CD) further streamlined updates, automated testing, and ensured that the application remained stable and up to date without manual intervention. These practices demonstrate the project's readiness for real-world use and its capacity for scalability and maintainability.

From a user perspective, the Streamlit dashboard provides an accessible and dynamic interface that allows real-time monitoring of sentiment trends. Despite some inherent limitations of the VADER model—such as limited handling of sarcasm and complex language nuances—the sentiment analysis delivered valuable directional insights that can support brand management, market research, and social listening efforts.

Looking ahead, this project serves as a robust foundation for expanding to additional social media platforms, incorporating more advanced natural language processing models, and enhancing the user experience with features such as personalized alerts, historical data analytics, and multilingual support. By continuing to evolve in these directions, the system could become an indispensable tool for businesses, researchers, and policymakers aiming to understand public sentiment and respond proactively to emerging trends.

In summary, the Cross-Platform Sentiment Analysis project showcases the effective combination of data engineering, machine learning, cloud computing, and user-centric design. It not only addresses the technical complexities involved in real-time multi-source data processing but also delivers actionable insights through a coherent and scalable platform. This project highlights the transformative potential of technology in making sense of the vast and ever-changing social media landscape and paves the way for future innovations in sentiment analysis and social data intelligence.

5.2 How it will make impact in market

The Cross-Platform Sentiment Analysis project holds significant potential to influence various sectors by enabling businesses, researchers, and decision-makers to better understand public opinion and trends in real-time across multiple social media platforms. Its impact in the market can be analyzed from several key perspectives:

Enhanced Brand Monitoring and Reputation Management

Brands and companies can leverage this system to continuously monitor how their products, services, or campaigns are perceived by consumers across Reddit, YouTube, and Tumblr. Real-time sentiment insights help identify positive feedback and emerging issues before they escalate, enabling timely responses to customer concerns. This proactive engagement improves customer satisfaction, loyalty, and overall brand reputation.

Informed Marketing and Advertising Strategies

By analyzing sentiment data across platforms, marketing teams gain a deeper understanding of audience reactions and preferences. This intelligence allows for more targeted campaigns, better messaging, and efficient resource allocation. Understanding public sentiment in real time also helps marketers adjust their strategies dynamically in response to market trends and competitor activity, resulting in increased campaign effectiveness and ROI.

Competitive Intelligence and Market Research

The ability to aggregate and analyze sentiment from multiple sources provides businesses with a comprehensive view of their competitors and industry landscape. It facilitates benchmarking, identifies emerging trends, and uncovers gaps or opportunities in the market. Researchers and analysts can use these insights to generate actionable reports that support strategic planning and innovation.

Crisis Detection and Management

Organizations can detect sudden negative sentiment spikes related to product failures, controversies, or misinformation by monitoring live social media data. Early detection enables swift crisis management, minimizing damage and restoring public trust more

effectively. This capability is crucial in today's fast-paced digital environment where news spreads rapidly.

Supporting Data-Driven Decision Making

By offering a real-time, data-driven perspective on public opinion, the platform empowers executives and decision-makers to base their actions on quantitative insights rather than intuition. This reduces risk, improves forecasting accuracy, and promotes evidence-based strategies across marketing, customer service, product development, and public relations.

Facilitating Research and Social Analysis

Beyond commercial uses, this tool can assist academic researchers, sociologists, and political analysts in studying social behavior, public sentiment on policy issues, and cultural trends. The ability to monitor multiple platforms simultaneously enriches the quality and scope of social research.

Scalability for Emerging Markets and Diverse Use Cases

As the platform expands to include more social media sources and languages, its market applicability will grow globally. It can be adapted for specific industries such as finance, healthcare, entertainment, and government agencies, tailoring sentiment analysis to sector-specific needs.

Cost-Effective and Accessible Solution

By utilizing open-source models like VADER and cloud infrastructure such as AWS EC2, the system offers a cost-effective alternative to expensive commercial sentiment analysis tools. The user-friendly Streamlit dashboard ensures accessibility for users without deep technical expertise, broadening its adoption potential.

APPENDIX

6.1 Code Snippet

.env file

reddit-api

- REDDIT_CLIENT_ID=obGQW4OVYaxr7T4rqaqgsw
- REDDIT_CLIENT_SECRET=20yFRLFtVV_9U7vk2gZtyH47WFFdig
- REDDIT_USER_AGENT=sentiment_scripts by /u/Reasonable-Listen516

tumblr-api

- TUMBLR_API_KEY=33Bhcnn3axYKVmgrptKIRDdPpiQC8Dh0flFRcCpesQtmYApi9u
- TUMBLR_API_SECRET=ZaNq8OBvsfxTv8jOehnq0DgNXPV4rHHLAQKPAkrnoVwxoMR1IH
- TUMBLR_ACCESS_TOKEN=lJVr2lqoMYGytDwszE5ICUcLubF4ffCAQABmlxan9CQALw8x7Q
- TUMBLR_ACCESS_TOKEN_SECRET= MRGjXmiQ6UBTSfnv4nvi07oDPjH5xSali04zX6E5cWuKFs5w61

#youtube-api

- YOUTUBE_API_KEY=AIzaSyAqDEW_z3bGg-qzZh9EaSSpDtivRw8oczc

Sentiment.py

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

def analyze_sentiment(text):

    score = analyzer.polarity_scores(text)

    compound = score['compound']

    if compound >= 0.05:

        return "Positive", compound

    elif compound <= -0.05:

        return "Negative", compound
```

```
    else:
        return "Neutral", compound
```

Main.py

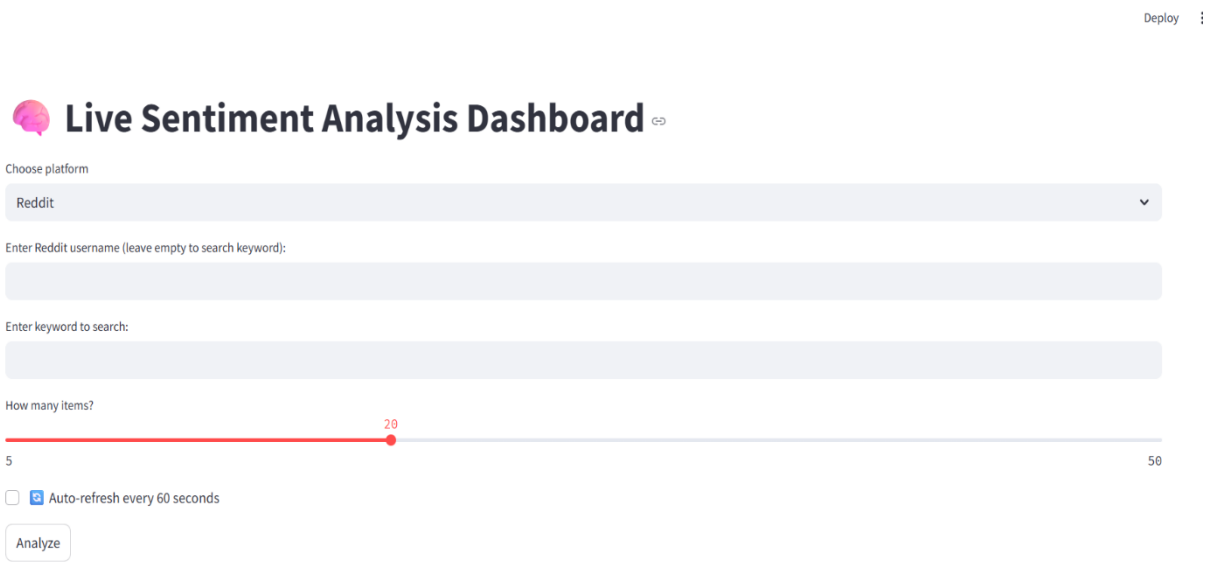
```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import time
import os
from dotenv import load_dotenv
dotenv_path = r"C:\Users\maukd\OneDrive\Pictures\ajay-project\.env"
load_dotenv(dotenv_path)
from reddit_api import fetch_reddit_comments_by_keyword,
fetch_reddit_comments_by_user
from youtube_api import fetch_youtube_comments, fetch_youtube_comments_by_channel
from tumblr_api import fetch_tumblr_posts, fetch_tumblr_posts_by_blog
from sentiment import analyze_sentiment
st.set_page_config(layout="wide")
st.title("□ Live Sentiment Analysis Dashboard")
platform = st.selectbox("Choose platform", ["Reddit", "YouTube", "Tumblr"])
if platform == "Reddit":
    username = st.text_input("Enter Reddit username (leave empty to search keyword):")
    if username:
        query = None
    else:
        query = st.text_input("Enter keyword to search:")
elif platform == "YouTube":
    username = st.text_input("Enter YouTube channel ID (leave empty if you want to input
video ID):")
```

```
    query = st.text_input("Enter video ID (if channel ID empty):")
elif platform == "Tumblr":
    username = st.text_input("Enter Tumblr blog name (leave empty to search by tag):")
    if not username:
        query = st.text_input("Enter tag to search:")
    else:
        query = None
limit = st.slider("How many items?", 5, 50, 20)
refresh = st.checkbox("🔄 Auto-refresh every 60 seconds")
last_updated = st.empty()
def analyze_platform(platform, username, query, limit):
    try:
        if platform == "Reddit":
            if username:
                return fetch_reddit_comments_by_user(username, limit)
            elif query:
                return fetch_reddit_comments_by_keyword(query, limit)
        elif platform == "YouTube":
            if username:
                return fetch_youtube_comments_by_channel(username, limit)
            elif query:
                return fetch_youtube_comments(query, limit)
        elif platform == "Tumblr":
            if username:
                return fetch_tumblr_posts_by_blog(username, limit)
            elif query:
                return fetch_tumblr_posts(query, limit)
    except Exception as e:
        st.error(f"Error fetching data: {e}")
```

```
    return []
def run_analysis():
    st.info("Fetching live data...")
    comments = analyze_platform(platform, username, query, limit)
if not comments:
        st.warning("No comments/posts found. Try a different input.")
        return
 results = []
    for text in comments:
        label, score = analyze_sentiment(text)
        results.append({"Text": text, "Sentiment": label, "Score": score})
  df = pd.DataFrame(results)
    st.dataframe(df)
  st.subheader("📊 Sentiment Distribution")
    sentiment_counts = df["Sentiment"].value_counts()
    fig, ax = plt.subplots()
    ax.pie(sentiment_counts, labels=sentiment_counts.index, autopct="%1.1f%%")
    st.pyplot(fig)
    last_updated.text(f"Last updated at {time.strftime('%H:%M:%S')}")
if st.button("Analyze") or refresh:
    while True:
        run_analysis()
        if not refresh:
            break
        time.sleep(60)
        st.experimental_rerun()
```

6.2 Snapshot

DashBoard:

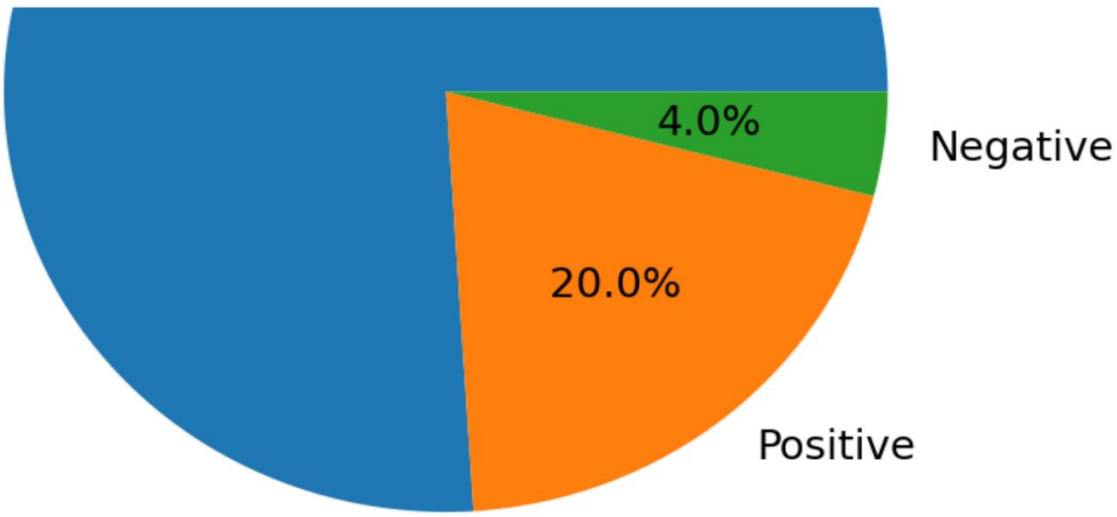


Sentimental Analysis:

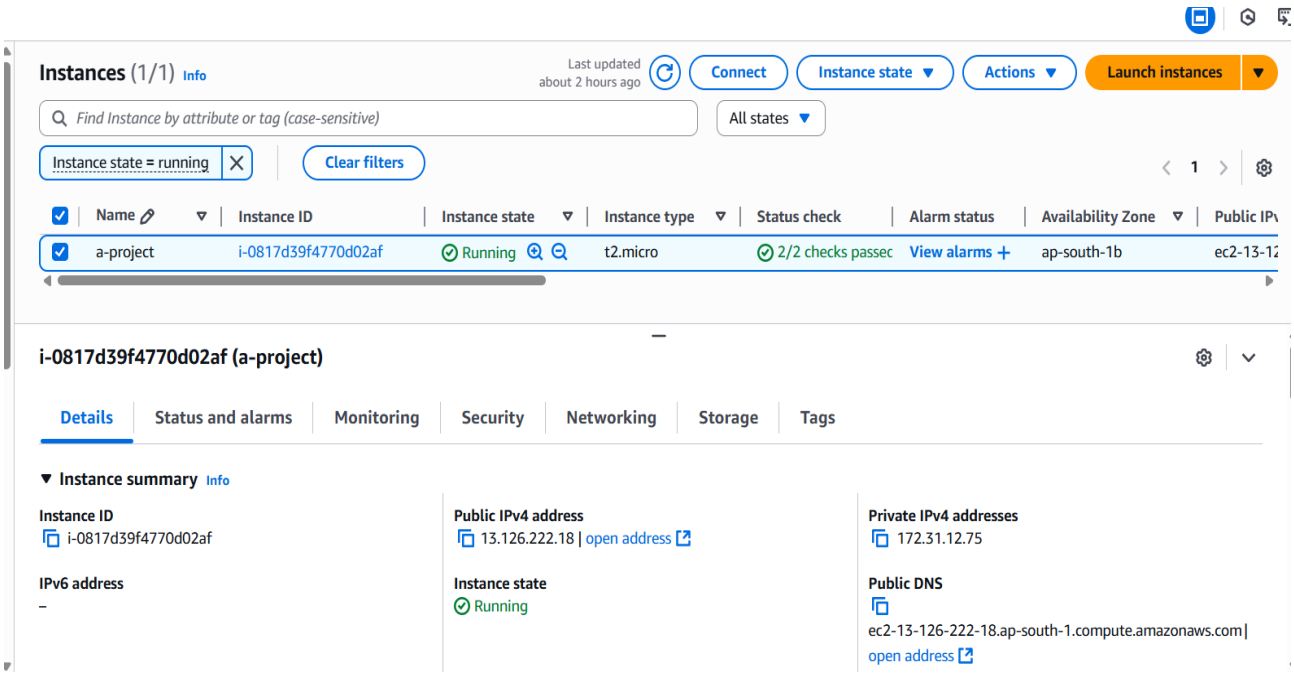
Analyze

Fetching live data...

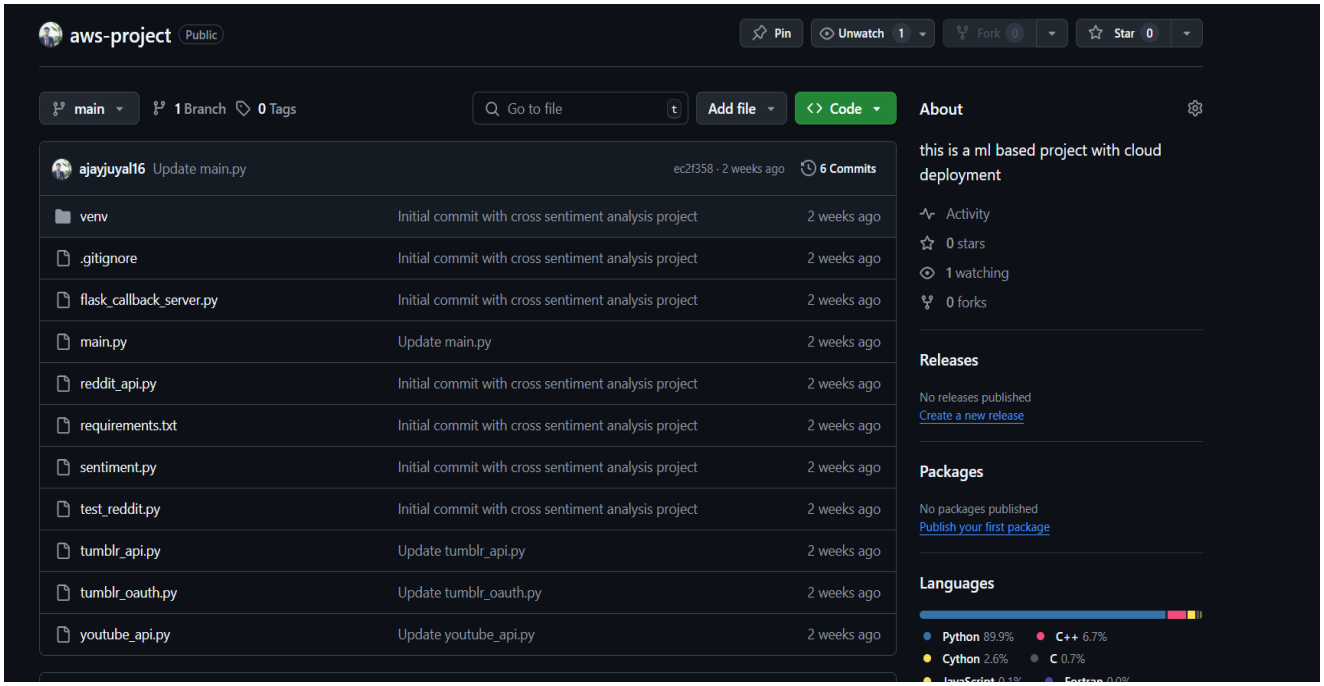
	Text	Sentiment	Score
2	Cricket player gets hit on his balls twice in the same game!	Neutral	0
3	Cricket match	Neutral	0
4	This is why cricket will never grow	Neutral	0
5	Today the US just pulled off one of the greatest WC upsets in history, and even got coverage on ESPN, which is insane. My	Negative	-0.8968
6	Any cricket fans here?	Neutral	0
7	A spectator invading cricket match in New York between India and Bangladesh.	Neutral	0
8	crazy good catch by cricketer.	Positive	0.128
9	[Highlight] A group of crickets throw out the first pitch at the Padres-Blue Jays game	Positive	0.34
10	Aside from Cricket, who is your favorite recurring character?	Positive	0.4588
11	For me, it's got to be the lawyer. He always starts out as the straight man, but inevitably goes a little bit off the deep end.	Positive	0.1154



AWS Deployment:



GitHub:

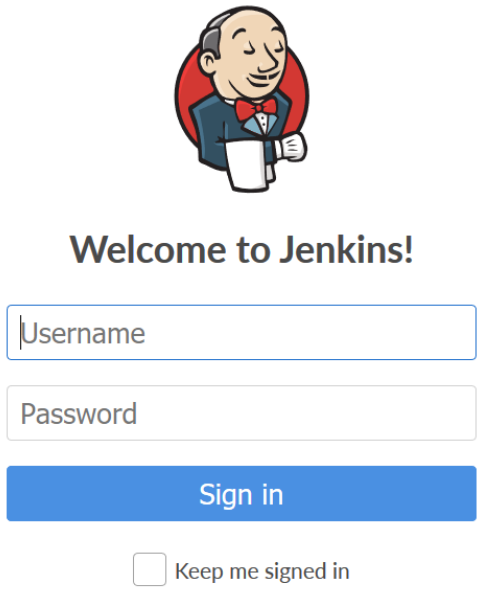


IDE:

aws Search [Alt+S]

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import time
import os
from dotenv import load_dotenv

dotenv_path = r"C:\Users\maukd\OneDrive\Pictures\ajay-project\.env"
load_dotenv(dotenv_path)

from reddit_api import fetch_reddit_comments_by_keyword, fetch_reddit_comments_by_user
from youtube_api import fetch_youtube_comments, fetch_youtube_comments_by_channel
from tumblr_api import fetch_tumblr_posts, fetch_tumblr_posts_by_blog
from sentiment import analyze_sentiment

st.set_page_config(layout="wide")
st.title("🔴 Live Sentiment Analysis Dashboard")

platform = st.selectbox("Choose platform", ["Reddit", "YouTube", "Tumblr"])

if platform == "Reddit":
    username = st.text_input("Enter Reddit username (leave empty to search keyword):")
    if username:
        query = None
```

CI/CD:



REFERENCES

7.1 Documentation and Official Resources

Programming Language & Core Tools

- Python (Official Documentation)
  https://docs.python.org/3/
- pip (Python Package Installer)
  https://pip.pypa.io/en/stable/

APIs & SDKs

- Reddit API (via PRAW)
  https://praw.readthedocs.io/
- YouTube Data API (v3)
  https://developers.google.com/youtube/v3
- Tumblr API (v2)
  https://www.tumblr.com/docs/en/api/v2

Natural Language Processing

- VADER Sentiment Analysis
  https://github.com/cjhutto/vaderSentiment
- NLTK (Natural Language Toolkit)
  https://www.nltk.org/

Web Dashboard Framework

- Streamlit (Python Web App Framework)
  https://docs.streamlit.io/

Containerization and DevOps

- Docker (Containerization Platform)
  https://docs.docker.com/
- Docker Hub (Image Repository)
  https://hub.docker.com/
- Jenkins (CI/CD Automation Server)
  https://www.jenkins.io/doc/

Cloud and Hosting

- Amazon EC2 (Elastic Compute Cloud)
  https://docs.aws.amazon.com/ec2/
- AWS CLI (Command Line Interface)
  https://docs.aws.amazon.com/cli/

Security & Environment Management

- python-dotenv (Environment Variable Loader)
  https://pypi.org/project/python-dotenv/

Version Control

- Git (Version Control System)
  https://git-scm.com/doc
- GitHub (Repository Hosting)
  https://docs.github.com/en

Visualization & Analysis

- Matplotlib (2D Graphs and Charts)
  https://matplotlib.org/stable/contents.html
- Seaborn (Statistical Data Visualization)
  https://seaborn.pydata.org/
- Pandas (Data Manipulation and Analysis)
  https://pandas.pydata.org/docs/